

Experiment No.13

Date:

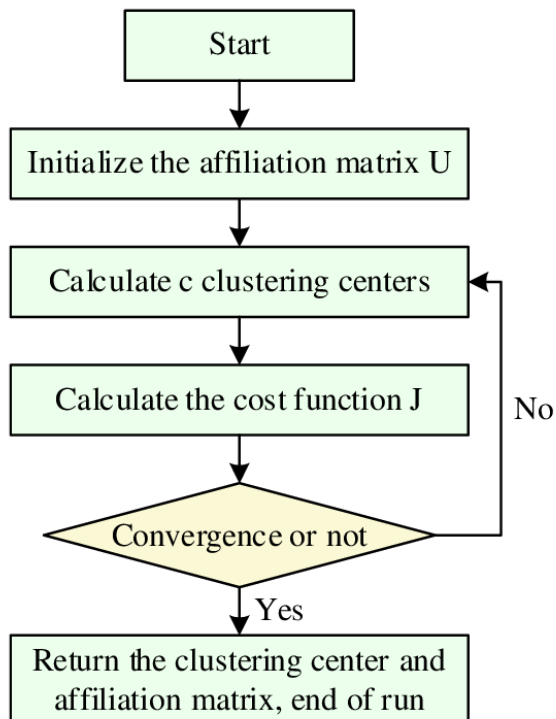
Fuzzy C-Means Clustering

Question : Write a program to demonstrate the use of Fuzzy C-Means Clustering

Aim : To demonstrate the use of Fuzzy C-Means Clustering

Explanation:

- The Fuzzy C-Means (FCM) algorithm is a soft clustering method where each data point can belong to multiple clusters with a certain degree of membership.
- It is a variation of the k-means algorithm that allows for data points with overlapping characteristics to be assigned to more than one group, unlike hard clustering which assigns a data point to only one cluster.
- FCM works by iteratively calculating cluster centers and updating the membership values of each data point to minimize an objective function.



Algorithm 1 The FCM algorithm

Input: A dataset X , clusters c , fuzzy factor e (usually take 2), stop iterating the condition ϵ , and maximum number of iterations T ;

Output: The clustering result C ;

- 1: Random initial clustering center c_j and initialize the membership matrix $u_{ij} = 0$;
- 2: In the iteration process, the membership matrix u_{ij} and the clustering center c_j are updated respectively according to Equations (4) and (3);
- 3: If Equation (2) or $t = T$ is satisfied, the iteration is stopped. Otherwise, set $t = t + 1$ and go back to step 2;
- 4: According to Equation (5), the classification of each sample point is determined;
- 5: Return C .

Minimize the following objective function:

$$J_m = \sum_{i=1}^n \sum_{j=1}^c (u_{ij})^m \cdot \|x_i - v_j\|^2$$

Where:

- n = number of data points
- c = number of clusters
- u_{ij} = membership of point i in cluster j
- $m > 1$ = fuzziness coefficient (controls overlap)
- v_j = cluster centroid
- $\|x_i - v_j\|$ = Euclidean distance between point and cluster center

Step 1: Initialization

- Define the number of clusters c
- Initialize a random **membership matrix** U of size $c \times n$
- Each **column sums to 1**, i.e., membership values for each point across all clusters sum to 1

Step 2: Centroid Calculation

- Calculate the centroid v_j of each cluster using the current membership values:

$$v_j = \frac{\sum_{i=1}^n (u_{ij})^m \cdot x_i}{\sum_{i=1}^n (u_{ij})^m}$$

- Weighted mean: points with higher membership have more influence

Step 3: Membership Update

- Update membership of each point for each cluster based on **inverse distance to centroids**:

$$u_{ij} = \frac{1}{\sum_{k=1}^c \left(\frac{\|x_i - v_j\|}{\|x_i - v_k\|} \right)^{\frac{2}{m-1}}}$$

- This ensures **closer points get higher membership** to the corresponding cluster

Step 4: Iteration

- Repeat **Steps 2 and 3** until:
 - Change in membership matrix U is smaller than a threshold (tolerance), or
 - Maximum number of iterations is reached

Step 5: Hard Clustering (Optional)

- Assign each point to the cluster with **highest membership**:

$$\text{cluster}(x_i) = \arg \max_j u_{ij}$$

Source Code:

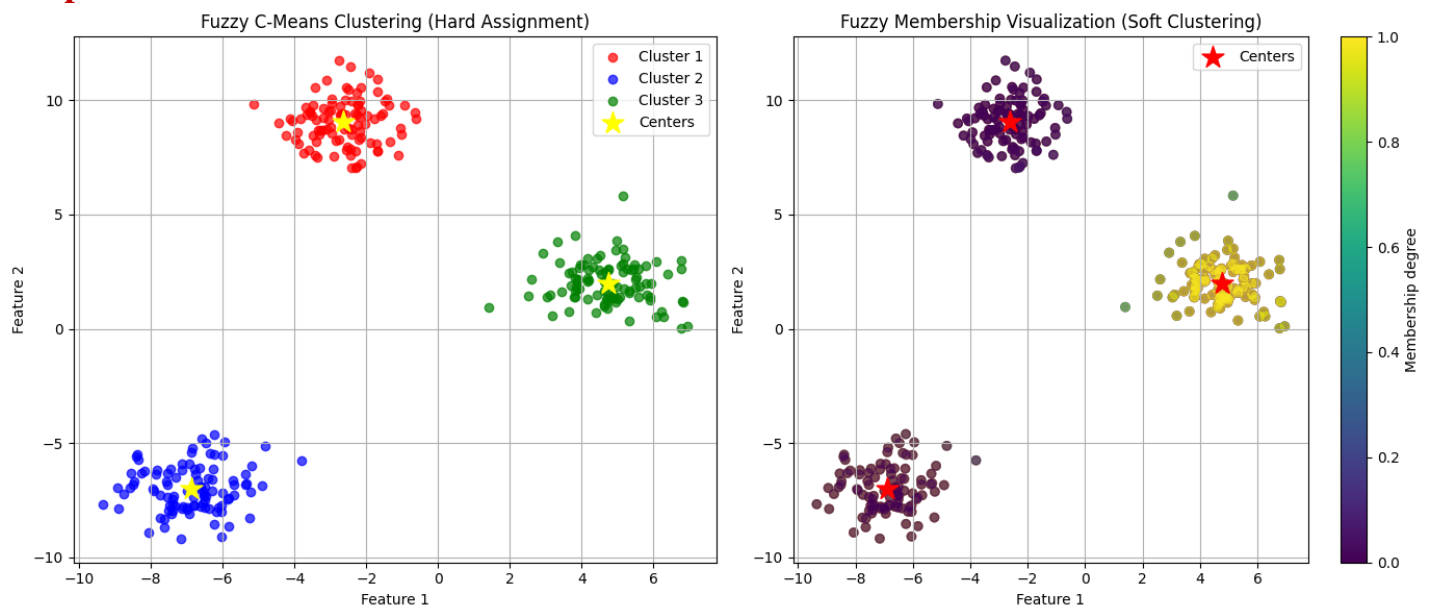
```

# Import libraries
import numpy as np
import matplotlib.pyplot as plt
import skfuzzy as fuzz
from sklearn.datasets import make_blobs
# Step 1: Generate sample data
n_samples = 300
n_features = 2
n_clusters = 3
X, y_true = make_blobs(n_samples=n_samples, centers=n_clusters,
n_features=n_features, random_state=42)
# Step 2: Transpose the data (required by scikit-fuzzy)
data = np.transpose(X)
# Step 3: Apply Fuzzy C-Means clustering
cntr, u, u0, d, jm, p, fpc = fuzz.cluster.cmeans(
    data=data,                # Data to cluster
    c=n_clusters,             # Number of clusters
    m=2,                      # Fuzziness parameter
    error=0.005,              # Convergence criterion
    maxiter=1000,             # Max number of iterations
    init=None                  # Random initialization
)
# Step 4: Get the cluster membership for each data point
cluster_membership = np.argmax(u, axis=0)
# Step 5: Plot hard cluster assignment
plt.figure(figsize=(14, 6))
plt.subplot(1, 2, 1)
colors = ['red', 'blue', 'green']
for j in range(n_clusters):
    plt.scatter(X[cluster_membership == j, 0],
                X[cluster_membership == j, 1],
                c=colors[j],
                label=f'Cluster {j+1}', alpha=0.7)
plt.scatter(cntr[:, 0], cntr[:, 1], marker='*', s=250, c='yellow',
label='Centers')
plt.title("Fuzzy C-Means Clustering (Hard Assignment)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.grid(True)
# Step 6: Soft clustering visualization
plt.subplot(1, 2, 2)
for j in range(n_clusters):
    plt.scatter(X[:, 0], X[:, 1], c=u[j], cmap='viridis', alpha=0.6)
plt.scatter(cntr[:, 0], cntr[:, 1], marker='*', s=250, c='red',
label='Centers')
plt.title("Fuzzy Membership Visualization (Soft Clustering)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")

```

```
plt.colorbar(label="Membership degree")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
# Step 7: Print clustering performance
print(f"Fuzzy Partition Coefficient (FPC): {fpc:.3f}")
```

Output:



Fuzzy Partition Coefficient (FPC): 0.958

Result: _____
